M0: запускаем API и проверяем базу

Начало практики: отдельная папка sneaker_shop и один проверяемый маршрут.

Наблюдение	Что доказывает
/docs открывается	Приложение отвечает
GET /health → 200	SQL-запрос к БД выполнен
БД остановлена → 503	Проверка не выдаёт ложный успех

Где что работает

Python + Uvicorn — на компьютере.
PostgreSQL — в Docker.

Результат главы

Файлы запуска, venv, Compose, конфигурация и GET /health.

↳ M0 принят, когда вы можете запустить всё по инструкции и показать отказ и восстановление БД.

11 Практика M0 · окружение

Шаг 1. Папка, Python и свой venv

Выберите команды для своей оболочки. Не смешивайте системный Python и окружение проекта.

macOS / Linux — Учебный пример

```
mkdir sneaker_shop
cd sneaker_shop
python3 --version
python3 -m venv .venv
source .venv/bin/activate
```

Windows · PowerShell — Учебный пример

```
mkdir sneaker_shop
cd sneaker_shop
py -3 --version
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
```

↳ Проверка: python -c "import sys; print(sys.executable)" показывает Python внутри .venv.

Шаг 2. Фиксируем файлы запуска

Конфигурация, зависимости и локальные секреты имеют разные роли.

Файл	Зачем он нужен
requirements.txt	Точные версии пакетов → повторяемая установка
.env.example	Шаблон переменных с вымышленными значениями
.env	Локальные значения; не добавляем в Git
.gitignore	.env, .venv/, __pycache__/ не попадают в коммит
app/core/config.py	Python загружает .env из корня проекта

После подготовки requirements.txt — Учебный пример

```
python -m pip install -r requirements.txt
python -m pip check
```

↳ В M0–M1 устанавливаем пакеты для запуска, БД и миграций. Зависимости авторизации добавим в M3.

Шаг 3. Просим настроить PostgreSQL

Промпт задаёт сервис, адрес, хранение данных и критерий готовности.

compose.yaml · фрагмент сервиса db — Учебный пример

```
db:
  image: postgres:17
ports:
  - "127.0.0.1:55433:5432"
volumes:
  - pgdata:/var/lib/postgresql/data
# environment и healthcheck
# см. полный compose.yaml
```

Что поручаем нейросети

Compose + .env.example + .gitignore.
База sneaker_shop; проверка pg_isready.

Что читаем в diff

Порт компьютера 55433.
Именованный том pgdata.
Пароль берётся из окружения.

↳ API с компьютера подключается к 127.0.0.1:55433. Адрес db:5432 относится к сети Compose.

Шаг 4. Поднимаем и проверяем БД

В корне проекта уже есть compose.yaml и заполненный .env.

Терминал · из корня проекта — Учебный пример

```
docker compose config --quiet
docker compose up -d --wait
docker compose ps
docker compose exec db psql \
  -U practice -d sneaker_shop \
  -c "SELECT 1;"
```

Что наблюдаем	Ожидаемый результат
Сервис db	running / healthy
Публикация порта	127.0.0.1:55433 → 5432
SELECT 1	Одна строка со значением 1

Следующая проверка

psql работает внутри контейнера.
Подключение Python проверим отдельно.

↳ Успешный запрос внутри контейнера подтверждает БД. Путь от FastAPI ещё предстоит проверить.

Шаг 5. API выполняет настоящий запрос

В полном примере Engine создаётся один раз; /health использует его соединение.

app/main.py · обработчик — Учебный пример

```
@app.get("/health")
def health():
    try:
        with engine.connect() as conn:
            conn.execute(text("SELECT 1"))
    except (OperationalError, TimeoutError):
        return JSONResponse(status_code=503,
            content={"status":
"unavailable"})
    return {"status": "ok"}
```

Запуск из корня проекта — Учебный пример

```
python -m uvicorn app.main:app \
  --reload --port 8000
```

Открыть в браузере

127.0.0.1:8000/docs
GET /health → Execute

↳ config.py читает адрес → database.py создаёт Engine → main.py выполняет SELECT 1.

Если M0 не запускается: проверяем слой

Идём от конкретного сообщения к одной проверке, затем повторяем запрос.

Симптом	Вероятная причина	Следующий шаг
ModuleNotFoundError	Другой Python / нет пакета	sys.executable; python -m pip show fastapi
address already in use	Порт занят другим процессом	Выбрать свободный порт и обновить URL
connection refused	БД не готова / неверный порт	compose ps; сверить 55433 в DATABASE_URL
password authentication failed	Реквизиты не совпадают	Сверить пользователя и пароль существующей БД
Таблиц ещё нет	M0 не создаёт схему	В M1 применить миграцию; не добавлять create_all

↳ Зафиксируйте папку, команду и ошибку. Исправьте одну причину и снова проверьте /health.

Принимаем M0: успех → отказ → успех

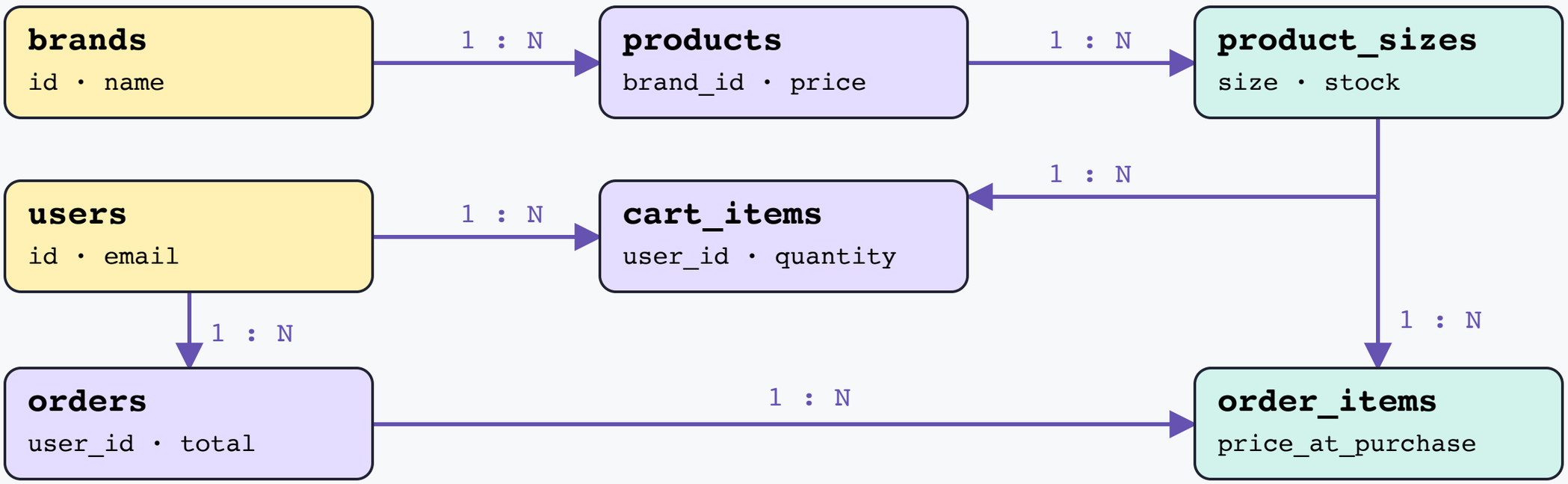
Эти ответы получены от настоящего Uvicorn и PostgreSQL, без подмены БД.

Действие	GET /health	JSON
БД работает	200	{"status":"ok"}
docker compose stop db	503	{"status":"unavailable"}
docker compose up -d --wait	200	{"status":"ok"}

↳ Остановку и восстановление выполняем в другом терминале; процесс API продолжает работать.

M1: сначала рисуем семь таблиц

Восстанавливаем связи до генерации кода. Стрелка: одна строка → много связанных строк.



У cart_items и order_items есть product_size_id. Остаток хранится в product_sizes; снимок цены — в order_items.

↳ Для каждого поля решаем: тип, NULL, значение по умолчанию, ключ и ограничение.

Шаг 1. Модели каталога

Попросите три класса: Brand, Product, ProductSize — на одном Base.

Product · фрагмент модели — Учебный пример

```
brand_id: Mapped[int] = mapped_column(
    ForeignKey("brands.id"))
price: Mapped[Decimal] = mapped_column(
    Numeric(10, 2))
brand: Mapped["Brand"] = relationship(
    back_populates="products")
```

Ограничения таблиц

price > 0 · stock ⩾ 0
UNIQUE(product_id, size)

Два разных механизма

ForeignKey — ссылка в БД.
relationship — переход в Python.

↳ Проверяем импорт трёх моделей и metadata. Таблицы появятся после Alembic upgrade.

Шаг 2. Пользователь и его корзина

Связи и ограничения описываем сейчас; вход и права API появятся в M3–M4.

CartItem · схема связей и Python — Учебный пример

```
user_id → ForeignKey("users.id")
product_size_id →
    ForeignKey("product_sizes.id")

UniqueConstraint("user_id",
    "product_size_id", name="uq_cart_size")
CheckConstraint("quantity > 0",
    name="ck_cart_quantity")
```

User

email — уникальный.
hashed_password — обязательный.
is_admin — false по умолчанию.

CartItem

Одна строка на пользователя и размер.
quantity не может быть 0.

↳ user_id связывает данные, но сам по себе не запрещает HTTP-запрос к чужой корзине.

Шаг 3. Заказ и позиции заказа

Разделяем общий документ и строки покупки. Цена одной пары фиксируется отдельно.

Order	OrderItem
user_id → users.id	order_id → orders.id
status = created	product_size_id → product_sizes.id
total: Numeric(12,2)	price_at_purchase: Numeric(10,2)
created_at с часовым поясом	quantity > 0

Связь в Python

order.items → список позиций.
item.order → один заказ.

Снимок цены

2 пары × 7 990 = 15 980.
price_at_purchase = 7 990.

↳ Модель описывает хранение. Расчёт total и транзакцию оформления реализуем в M5.

Шаг 4. Модели превращаем в миграцию

Пустая учебная БД + импорт всех моделей → кандидат ревизии → чтение → применение.

1 **Подключить metadata**

```
from app.models import Base
target_metadata = Base.metadata
```

2 **Создать кандидата**

```
python -m alembic revision --
autogenerate -m initial
```

3 **Прочитать файл**

```
7 create_table • поля • FK
UNIQUE • CHECK • nullable
```

4 **Применить и сверить**

```
python -m alembic upgrade head
python -m alembic current
```

↳ revision создаёт файл. upgrade меняет БД. В готовом эталоне ревизия 0001 уже есть.

Шаг 5. Seed заполняет каталог

Фиксированные данные помогают сравнить результат. Повторный запуск не сбрасывает остатки.

После миграции · два запуска — Учебный пример

```
python -m app.seed
python -m app.seed
```

Таблица	После 1-го	После 2-го
brands	2	2
products	3	3
product_sizes	4	4

Порядок вставки

Бренды → товары → размеры.
North Runner: 42 → 3, 43 → 0.

Чего seed не меняет

Существующие цены и остатки.
Пользователи и заказы пока пусты.

↳ 25 проверок M1 включают повтор seed, сохранение изменённого остатка и корректную генерацию новых ID.

Шаг 6. Проверяем с нуля и на ошибках

Новая отдельная база доказывает воспроизводимость миграции и seed.

Автопроверка готового эталона — Учебный пример

```
python verify.py
```

Что делает сценарий

Создаёт временную БД.
upgrade → seed × 2 → проверки.
Удаляет только свою временную БД.

Проверяем	Фактический результат
Схема	7 таблиц + alembic_version
Каталог	2 бренда / 3 товара / 4 размера
ORM-переход	North Runner → North → 42, 43
Ошибочные записи	12 отказов ограничений
Повтор и ID	Нет дубликатов; новый ID свободен

↳ Проверка на вашей старой БД не заменяет запуск с нуля. Рабочую базу и том для этого не удаляем.

Принимаем M1: объясни три результата

Схема создаётся миграцией, данные — seed, код обращается к ним через ORM.

Модель

Описывает таблицу в Python. Правка класса сама не меняет БД.

Миграция

Применяет изменения схемы. Создаёт таблицы и ограничения.

Seed

Добавляет данные каталога. Повтор сохраняет строки и stock.

Приёмка: upgrade с нуля ✓ · 7 таблиц ✓ · каталог 2/3/4 ✓ · повтор seed ✓ · отрицательные проверки ✓

↳ Перед M2 сохраните код, команды запуска и фактические результаты. Умейте объяснить каждый шаг.